

Orleans：用于可编程性与可扩展性的分布式虚拟 Actor (中文机器翻译稿)

原文：Bernstein 等，Microsoft Research MSR-TR-2014-41。正文按英文 PDF 分页翻译；插图取自对应英文页中的图块区域渲染截图。本文为机器翻译，仅供学习参考。

— 第 1 页 (英) / 译文 —

1 奥尔良：实现可编程性和可扩展性的分布式虚拟参与者 Philip A. Bernstein、Sergey Bykov、Alan Geller、Gabriel Kliot、Jorgen Thelin 微软研究院 摘要 大规模交互式服务需要高吞吐量、低延迟和高可用性，这是传统无状态三层架构难以满足的目标。Actor 模型使得构建有状态的中间层并实现所需的性能变得很自然。然而，流行的 Actor 模型平台仍然将许多分布式系统问题传递给开发人员。Orleans 编程模型引入了虚拟参与者的新颖抽象，解决了许多复杂的分布式系统问题，例如可靠性和分布式资源管理，将开发人员从处理这些问题中解放出来。同时，Orleans 运行时使应用程序能够获得高性能、可靠性和可扩展性。本文介绍了 Orleans 背后的设计原则，并演示了 Orleans 如何实现满足这些目标的简单编程模型。我们描述了 Orleans 如何简化 Windows Azure 上多个可扩展生产应用程序的开发，并报告这些生产系统的性能。1. 简介 构建可扩展且可靠的交互式服务非常困难。交互性对可用性和延迟施加了严格的限制，因为这直接影响最终使用的体验。为了支持大量并发用户会话，高吞吐量至关重要。具有无状态前端、无状态中间层和存储层的传统三层架构由于每个请求都必须参考存储层的延迟和吞吐量限制，因此可扩展性有限。通常在中间层和存储之间添加缓存层以提高性能[9][14][19]。然而，缓存失去了底层存储层的大部分并发性和语义保证。为了防止缓存项的并发更新导致不一致，应用程序或缓存管理器必须实现并发控制协议[11]。无论有没有缓存，无状态中间层都不提供数据局部性，因为它使用数据传送范例：对于每个请求，数据从存储或缓存发送到正在处理请求的中间层服务器。社交图的出现，其中单个请求可能会触及通过多跳关系动态连接的许多实体，这使得满足所需的应用程序级语义和缓存上的一致性以及交互式访问的快速响应变得更具挑战性。参与者模型通过依赖功能传送范例为这些挑战提供了一个有吸引力的解决方案。Actor 允许构建一个有状态的中间层，该中间层具有数据局部性缓存的性能优势以及通过特定于应用程序的操作封装实体的语义和一致性优势。此外，参与者可以轻松实现中间层实体之间的横向“社交”关系。分布式系统可编程性的另一种观点是通过面向对象编程 (OOP) 视角的视角。虽然 OOP 是一种对复杂系统进行建模的直观方法，但它已被流行的面向服务的架构 (SOA) 边缘化。在实现服务组件时，人们仍然可以从 OOP 中受益。然而，在系统级别，开发人员必须考虑松散耦合的分区服务，这通常与应用程序的概念对象不匹配。这导致了主流开发者构建分布式系统的难度。参与者模型将 OOP 带回到系统级别，参与者对于开发人员来说非常类似于熟悉的交互对象模型。Erlang [3] 和 Akka [2] 等 Actor 平台在简化分布式系统编程方面向前迈出了一步。然而，由于提供的抽象和系统服务水平相对较低，它们仍然给开发人员带来了许多分布式系统复杂性的负担。关键挑战是需要管理应用程序代码中参与者的生命周期

处理固有的分布式竞赛、处理参与者故障和恢复的责任、参与者的放置以及分布式资源管理。为了针对应用程序中的此类问题构建正确的解决方案，开发人员必须是分布式系统专家。为了避免这些复杂性，我们构建了 Orleans 编程模型和运行时，这提高了 Actor 的抽象级别。Orleans 的目标客户不是分布式系统专家的开发人员，尽管我们的专家客户也发现它很有吸引力。它是基于参与者的，但与现有的基于参与者的平台不同，它将参与者视为虚拟实体，而不是物理实体。首先，奥尔良演员实际上总是存在的。它不能被显式地创建或销毁。它的存在超越了其任何内存实例化的生命周期，因此也超越了任何特定服务器的生命周期。二、Orleans actor 自动实例化：如果内存中没有

— 第 2 页 (英) / 译文 —

2 个 actor 的实例，发送到 actor 的消息会导致在可用服务器上创建一个新实例。未使用的 Actor 实例将作为运行时资源管理的一部分自动回收。Actor 永远不会失败：如果服务器 S 崩溃，发送到在 S 上运行的 Actor A 的下一条消息会导致 Orleans 自动在另一台服务器上重新实例化 A，从而消除应用程序监督和显式重新创建失败的 Actor 的需要。第三，参与者实例的位置对于应用程序代码是透明的，这大大简化了编程。第四，Orleans 可以自动创建同一无状态 Actor 的多个实例，无缝扩展热门 Actor。总的来说，Orleans 为开发人员提供了一个虚拟的“参与者空间”，类似于虚拟内存，允许他们调用系统中的任何参与者，无论它是否存在于内存中。虚拟化依赖于从虚拟参与者映射到当前正在运行的物理实例的间接映射。这种间接级别为运行时提供了解决许多必须由开发人员解决的硬分布式系统问题的机会，例如 Actor 放置和负载平衡、未使用的 Actor 的停用以及服务器故障后 Actor 的恢复，这些问题对于他们来说很难解决。因此，虚拟参与者方法显著简化了编程模型，同时允许运行时透明地平衡负载并从故障中恢复。运行时支持通过分布式目录进行间接访问。Orleans 通过使用从参与者身份映射到其当前物理位置的本地缓存来最小化间接的运行成本。事实证明，这一策略非常有效。我们的生产服务中的缓存命中率通常远高于 90%。Orleans 已用于构建当前在 Microsoft Windows Azure 云上运行的多个生产服务，包括一些流行游戏的后端服务。这使我们能够验证使用 Orleans 编写的生产应用程序的可扩展性和可靠性，并根据此反馈调整其模型和实现。它还使我们能够验证（至少在传闻中）奥尔良编程模型可以显著提高程序员的生产力。虽然 Orleans 编程模型适用于许多应用程序，但某些模式不太适合 Orleans。一种这样的模式是一种将许多实体上的频繁批量操作与单个实体上的操作混合在一起的应用程序。参与者的隔离使得这种批量操作比共享内存数据结构上的操作更加昂贵。如果系统中的参与者数量非常大（数十亿）并且不存在时间局部性，则虚拟参与者模型可能会降级。Orleans 尚不支持跨角色事务，因此在数据库系统之外需要此功能的应用程序不适合。总之，本文的主要贡献是 (a) 一种新颖的虚拟参与者抽象，可以实现简化的编程模型；(b) 分布式参与者模型的高效且可扩展的实现，消除了传统参与者框架的一些编程复杂性，并具有良好的性能和可扩展性；(c) 详细测量和我们生产经验的描述。论文概要如下。在第 2 节中，我们介绍 Orleans 编程模型。第 3 节描述了运行时，重点是虚拟参与者模型如何实现可扩展性和可靠性。第 4 节讨论了如何在实践中使用 Orleans，第 5

节介绍了对生产基准和合成基准的测量。第 6 节将 Orleans 与其他 Actor 框架以及 [5] 中报告的 Orleans 早期原型进行比较。第 7 节是结论。2. 编程模型 本节描述 Orleans 编程模型并提供 Halo 4 Presence 服务的代码示例（在 4.1 节中进一步描述）。2.1 虚拟参与者 Orleans 编程模型基于 .NET Framework 4.5 [10]。演员是奥尔良的基本组成部分

应用程序和是隔离和分布的单位。每个参与者都有一个唯一的身份，由其类型和主键（128 位 GUID）组成。与任何对象一样，参与者封装了行为和可变状态。可以使用内置的持久性工具来存储其状态。Actor 是孤立的，也就是说，他们不共享内存。因此，两个参与者只能通过发送消息进行交互。奥尔良的演员虚拟化有四个方面：1. 永久存在：演员是虚拟地永远存在的纯逻辑实体。Actor 无法显式创建或销毁，并且其虚拟存在不受执行它的服务器故障的影响。由于参与者始终存在，因此他们始终是可寻址的。2. 自动实例化：Orleans 的运行时会自动在内存中创建名为激活的 Actor 实例。在任何时间点，参与者都可能有一个或多个激活。如果没有待处理的请求，演员将不会被实例化。当新请求发送到当前未实例化的 Actor 时，Orleans 运行时会通过选择服务器、在该服务器上实例化实现 Actor 的 .NET 对象并调用其 `ActivateAsync` 方法进行初始化来自动创建激活。如果当前实例化 Actor 的服务器发生故障，运行时将在下次调用时自动在新服务器上重新实例化它。这意味着 Orleans 不需要像 Erlang [3] 和 Akka [2] 中那样的监督树，其中应用程序负责重新创建失败的 Actor。未使用的演员的记忆

— 第 3 页 (英) / 译文 —

3 实例作为运行时资源管理的一部分自动回收。执行此操作时，Orleans 会调用 `DeactivateAsync` 方法，该方法使参与者有机会执行清理操作。3. 位置透明性：一个 actor 可能在不同的时间在不同的位置被实例化，有时可能根本没有物理位置。与参与者交互或在参与者内运行的应用程序不知道参与者的物理位置。这类似于虚拟内存，其中给定的逻辑内存页可以随着时间的推移被映射到各种物理地址，并且有时可能被“调出”并且不映射到任何物理地址。正如操作系统自动从磁盘调入内存页面一样，Orleans 运行时会根据新请求自动实例化一个未实例化的 Actor 4. 自动横向扩展：目前，Orleans 支持 Actor 类型的两种激活模式：单一激活模式（默认），其中只允许同时激活一个 Actor，以及无状态工作模式，其中 Orleans 按需自动创建一个 Actor 的许多独立激活（最多一个限制）以提高吞吐量。“独立”意味着同一参与者的不同激活之间不存在状态协调。因此，无状态工作模式适合具有不可变或无状态的 Actor，例如充当只读缓存的 Actor。让参与者成为虚拟实体，而不是物理实体，对奥尔良编程模型和实现具有重大影响。自动激活、位置透明和永久存在极大地简化了编程模型，因为它们从应用程序中消除了显式激活或停用参与者、监督其生命周期以及在失败时重新创建它的需要。2.2 Actor 接口 Actor 通过声明为其强类型接口一部分的方法和属性来相互交互。Actor 接口的所有方法和属性都要求是异步的；也就是说，它们的返回类型必须是 `Promise`（参见第 2.4 节）。(1) 公共接口 `IGameActor : IActor` (2) { (3) `Task<string> GameName { get; }` (4) 任务 `<List<IPlayerActor>> CurrentPlayers { 获取; }` (5) 任务 `JoinGame(IPlayerActor game);` (6) 任务 `LeaveGame(IPlayerActor 游戏);` (7) } 2.3 Actor 引用 Actor 引用是一个强类型的虚拟 Actor 代理，它允许其他 Actor 以及非 Actor 代码调用其上的方法和属性。可以通过调用工厂类的 `GetActor` 方法（Orleans 在编译时自动生成）并指定 actor 的主键来获取 actor 引用。还可以从远程方法或属性返回接收引用。参与者引用可以作为输入参数传递给参与者方法调用。(1) `public static class GameActorFactory` (2) { (3) `public static IGameActor GetActor(Guid gameId);` (4) } 演员参考是虚拟的。演员引用不会向程序员公开目标演员的任何位置信息。它也没有绑定的概念。在传统的 RPC 模型（例如 Java RMI、CORBA 或 WCF）中，程序员需要显式地将虚拟引用绑定到服务，通常通过外部注册表或位置服务。在奥尔良，参与者引用由发送者在本地创建，并且可以立即使用，无需绑定或注册步骤。通过允许立即将请求传送到参与者而无需等待绑定或解析服务端点，从而简化了编程并最大化了吞吐量。2.4 Promises Actor 通过发送异步消息进行交互。与大多数现代分布式系统编程模型一样，这些消息交换作为方法调用公开。然而，与传统模型不同的是，Orleans 方法调用立即返回并承诺将来的结果，而不是阻塞直到返回结果。`Promise` 允许并发而不需要显式的线程管理。`Promise` 具有三种状态的生命周期。最初，一个承诺是未兑现的；它代表了在未来某个时间收到结果的期望。当结果是

收到后，承诺就得到履行，结果就成为承诺的价值。如果在结果计算或通信过程中发生错误，承诺就会被打破。`Promise` 作为 `System.Threading.Tasks.Task<T>` 类的实例公开，该类表示指定类型的最终值，或作为 `System.Threading.Tasks.Task` 类的实例（表示与 `void` 方法相对应的完成承诺）。使用 `Promise` 的主要方法是安排一个闭包（或延续）在 `Promise` 解决时执行。通常通过在 `Promise` 上使用 `Await C#` 关键字来隐式调度闭包。在下面的示例中，编译器执行堆栈撕裂，并将“await”之后的代码转换为在 `Promise` 解决后执行的闭包。因此，开发人员编写的代码（包括错误处理）异步执行，但看起来是顺序的，因此更自然。(1) `IGameActor gameActor = GameActorFactory.GetActor(gameId);` (2) `try{` (3) 字符串名称 = `await gameActor.GameName;` (4) `Console.WriteLine(“游戏名称是”+名称);` (5) `}catch(Exception){` (6) `Console.WriteLine(“对 actor 的调用失败”);` (7) }

— 第 4 页 (英) / 译文 —

4.2.5 轮次 Actor 激活是单线程的，并以块的形式工作，称为轮次。激活一次执行一回合。轮次可以是方法调用或在承诺解决时执行的闭包。虽然奥尔良可以并行执行不同激活的轮次，但每个激活总是一次执行一轮。基于回合的异步执行模型允许对同一激活的多个请求进行交替回合。由于推理多个请求的交错执行具有挑战性，默认情况下，Orleans 通过等待激活完成处理一个请求，然后再分派下一个请求来避免这种交错。因此，在处理当前请求期间创建的所有 `Promise` 都已得到解决并且所有关联的闭包都已执行之前，激活不会收到新请求。要覆盖此默认行为，参与者类可以使用 `[Reentrant]` 属性进行标记。这表明该类的激活可能会在前一个请求的轮次之间给出另一个请求来处理，例如在等待挂起的 IO 操作时。两个请求的轮次执行仍然保证是单线程的，因此激活仍然一次执行一轮。但是属于可重入参与者的不同请求的轮次可以自由地交错。2.6 持久性 Actor 请求的执行可能会修改 Actor 状态，该状态可能是持久的，也可能不是持久的。Orleans 提供了简化持久性管理的

工具。一个actor类可以声明一个属性包接口来表示应该被持久化的actor状态。然后，运行时为该类型的每个参与者提供一个状态对象，该对象实现该接口并公开用于保存和刷新状态的方法。(1) // 状态属性包接口 (2) public interface IGameState : IState (3) { (4) GameState Status { get; set; } (5) List<IPlayerActor> 玩家 { get; set; } (6) } (7) (8) // Actor 类实现 (9) public class GameActor : ActorBase<IGameState> IGameActor (10) { (11) Task JoinGame(IPlayerActor game) (12) { (13) // 更新状态属性包 (14) this.State.Players.Add(IPlayerActor); (15) // 检查点 actor 状态 (16) return this.State.WriteStateAsync(); (17) } (18) } 何时对 Actor 的持久状态进行检查点取决于应用程序逻辑。例如，它可以在每个应用程序请求完成时执行此操作，或者基于计时器或基于自上一个检查点以来处理的请求数量定期执行此操作。与底层存储的交互是通过持久性提供者实现的，持久性提供者充当特定存储系统的适配器：SQL 数据库、列存储、blob 存储等。2.7 定时器和提醒器 Orleans 有两种定时器设施。瞬态定时器非常模仿 .NET 定时器接口，但提供单线程执行保证。它们是在 Actor 激活的本地创建的，并在 Actor 停用消失。提醒是一个计时器，无论演员是否活跃都会触发。因此，它超越了创建它的参与者激活，并继续运行直到被显式删除。如果在参与者未激活时触发提醒，则会自动创建新的激活来处理提醒消息，就像发送到该参与者的任何其他消息一样。提醒器是可靠的持久计时器，它为创建它们的参与者生成消息，同时允许运行时通过在提醒滴答之间停用这些参与者来回收系统资源（如有必要）。提醒遵循虚拟参与者的概念模型，虚拟参与者永久存在于系统中，并且仅在需要处理传入请求时在内存中激活。提醒是一种有用的工具，可以在失败的情况下执行不频繁的定期工作，并且无需将参与者的激活永远固定在内存中。3. 运行时实现 在本节中，我们描述运行时的总体架构，重点介绍关键设计选择及其基本原理。我们的指导原则是在不牺牲性能的情况下实现简单的编程模型。3.1 概述 Orleans 运行在一个服务器集群上

数据中心，每个数据中心都运行一个容器进程来创建和托管参与者激活。服务器具有三个关键子系统：消息传递、托管和执行。消息传递子系统使用单个 TCP 连接连接每台服务器，并使用一组通信线程通过开放连接在这些服务器上托管的参与者之间多路复用消息。托管子系统决定在哪里放置激活并管理它们的生命周期。执行子系统在一组具有单线程和可重入保证的计算线程上运行参与者的应用程序代码。当一个参与者调用另一个参与者时，Execution 会将方法调用转换为消息，并将其与目标参与者的身份一起传递给 Messaging。消息传递与托管协商以确定要将消息发送到的目标服务器。托管维护一个分布式目录来跟踪集群中的所有参与者激活。它要么找到目标参与者的现有激活，要么选择服务器来创建新的激活

— 第 5 页 (英) / 译文 —

其中 5 个。然后消息传递序列化消息并将其发送到已打开的套接字到目标服务器。在接收端，调用参数被反序列化并封装到一组强类型对象中，然后传递给 Execution，由 Execution 安排调用。如果参与者正忙于处理先前的调用，则该请求将排队，直到该请求的执行完成。如果接收服务器被指示创建一个新的激活，它会在目录中注册参与者，然后创建它的本地内存实例。单次激活保证由目录提供。托管还负责本地管理服务器中的资源。如果参与者在可配置的时间内处于空闲状态或服务器遇到内存压力，则运行时会自动停用它并回收其系统资源。这种简单的本地资源管理策略是通过参与者虚拟化实现的。未使用的 Actor 可以在任何服务器上独立地本地停用和回收，因为稍后可以透明地重新激活它。这种方法不需要复杂的分布式垃圾收集协议，该协议涉及在回收参与者之前跟踪对参与者的所有物理引用。3.2

分布式目录许多分布式系统使用确定性放置来避免通过一致散列或基于范围的分区来维护每个组件位置的显式目录。Orleans 允许完全灵活的放置，将每个参与者的位置保留在分布式目录中。这使得运行时可以在系统负载发生变化时通过放置和移动参与者来更自由地管理系统资源。Orleans 目录被实现为一跳分布式哈希表 (DHT) [17]。集群中的每个服务器都保存目录的一个分区，并且使用一致哈希将参与者分配给这些分区。目录中的每条记录都将一个参与者 ID 映射到其激活的位置。当创建新的激活时，注册请求将被发送到适当的目录分区。类似地，当激活被停用时，会向分区发送请求以取消注册激活。单激活约束由目录强制执行：如果收到已注册激活的单激活参与者的注册请求，则新注册将被拒绝，并且现有激活的地址将随拒绝一起返回。使用分布式目录进行放置和路由意味着每条消息都会有一个额外的跃点，以找出目标参与者的物理位置。因此，Orleans

在每台服务器上维护一个大型本地缓存，其中包含最近解析的参与者到激活的映射。单激活 Actor 的每个缓存条目大约为 80 字节。这使我们能够轻松地在典型的生产服务器上缓存数百万个条目。我们在生产中发现缓存具有非常高的命中率，并且足够有效，几乎完全消除了对每条消息进行额外跳跃的需要。3.3 强隔离 奥尔良的 Actor 不共享状态，并且彼此隔离。Actor 进行通信的唯一方式是发送消息，这些消息作为 Actor 引用上的方法调用公开。在这方面，奥尔良遵循标准参与者范式。此外，方法调用参数和返回值在参与者调用之间同步深度复制，即使两个参与者碰巧驻留在同一台机器上，以保证发送数据的不变性。为了降低深度复制的成本，奥尔良使用了两种互补的方法。首先，应用程序可以通过在 actor 方法签名中使用标记通用包装类 `Immutable<T>` 来指定它不会改变参数。这告诉运行时不复制参数是安全的。当参与者代码从不改变参数时，这对于直通函数式场景非常有用。此类功能的一个示例是 Halo 4 存在服务中的 Router actor (第 4.1 节)，它对传递的数据 blob 执行解压缩，而不存储或改变它。对于案例

在必须进行实际复制的地方，Orleans 使用高度优化的复制模块，该模块是序列化子系统的一部分 (下面的 3.7 节)。3.4 异步 Orleans 强加了异步编程风格，使用 Promise 来表示未来的结果。所有对 actor 方法的调用都是异步的；结果必须是 Task 或 Task<T> 类型，以指示稍后将解决它们。.NET 4.5 中引入的异步编程模型基于 async 和 await 关键字，极大地简化了处理 Promise 的代码。Orleans 普遍使用异步对于应用程序的简单性和可扩展性非常重要。防止应用程序代码在等待结果时占用线程可确保系统吞吐量受远程请求成本的影响最小。在我们的测试中，增加的分布会因更多的离机调用而导致更高的延迟，但对通信密集型应用程序的吞吐量几乎没有影响。3.5 单线程 Orleans 确保每次激活中一次最多运行一个线程。因此，激活状态永远不会被多个线程同时访问，因此竞争条件是不可能的，并且锁和其他同步原语是不必要的。此保证由执行子系统提供，无需创建每次激活线程。虽然单线程确实限制了单个激活的性能，但处理不同请求的多个激活之间的并行性是

— 第 6 页 (英) / 译文 —

6 足以有效地使用可用的 CPU 资源，并且实际上会带来更好的整体系统响应能力和吞吐量。3.6 协作式多任务处理 Orleans 使用协作式多任务处理来安排应用程序轮流。这意味着应用程序一旦启动，就会不间断地运行直至完成。Orleans 调度程序使用它控制的少量计算线程（通常等于 CPU 核心的数量）来执行所有应用程序参与者代码。为了支持服务器上数万到数百万的参与者，每次激活都需要一个线程的抢占式多任务处理将需要比现代硬件和操作系统所能支持的更多线程。即使线程数量没有超过实际限制，由于上下文切换的开销和丢失的缓存局部性，数千个线程的抢占式多任务处理的性能也很差。通过仅使用协作多任务处理，Orleans 可以在少量线程上高效地运行大量激活。协作多任务处理还允许 Orleans 应用程序以非常高的 CPU 利用率运行。我们已经在 25 台服务器完全饱和的情况下以 90% 以上的 CPU 利用率运行负载测试，没有任何不稳定的情况。协作多任务处理的一个弱点是，表现不佳的组件可能会占用整个处理器，从而降低其他组件的性能。对于奥尔良来说，这不是一个主要问题，因为所有演员都属于同一开发商。（Orleans 目前不适用于多租户环境。）Orleans 确实提供了对长时间运行轮次的监控和通知，以帮助故障排除，但我们通常不认为这是生产中的问题。3.7 序列化 将复杂对象编组为字节流，然后重新创建对象是任何分布式系统的核心部分。虽然此过程对应用程序开发人员是隐藏的，但其效率会极大地影响整体系统性能。Protocol Buffers [12] 等序列化包提供了出色的性能，但代价是限制了可能传递的对象类型。许多序列化器不支持动态类型或任意多态性，并且许多不支持对象标识（以便指向同一对象的两个指针在反序列化后仍然指向同一对象）。标准 .NET 二进制文件支持任何标有 [Serialized]

属性的类型，但速度很慢，并且可能会创建非常大的表示形式。为了更好的可编程性，Orleans 允许任何数据类型并通过序列化器维护对象标识。可以使用结构体、数组、完全多态和通用对象。我们通过高度优化的序列化子系统来平衡这种灵活性，该子系统可与“标准”类型上可用的最佳子系统竞争。我们通过在编译时自动生成自定义序列化代码以及针对常见类型（例如 .NET 集合）手工编写的代码来实现此目的。序列化表示很紧凑，并且携带最少量的动态类型信息。3.8 可靠性 Orleans

自动管理可靠性的各个方面，使程序员无需显式地这样做。Orleans 唯一不管理的方面是参与者的持久状态：这部分留给开发人员。Orleans 运行时有一个用于管理服务器的内置成员资格机制。服务器通过定期心跳自动检测故障，并就成员资格视图达成一致。在发生故障后的短时间内，不同服务器上的成员资格视图可能会有所不同，但可以保证最终所有服务器都会了解发生故障的服务器并具有相同的成员资格视图。收敛时间取决于故障检测设置。使用 Orleans 的生产服务配置为检测故障并在 30 到 60 秒内聚合集群成员资格。此外，如果会员服务宣布服务器死亡，即使故障只是暂时的网络问题，它也会自行关闭。当服务器发生故障时，该服务器上的所有激活都会丢失。如果发生故障，则故障服务器上的目录信息将丢失

不复制目录分区。一旦幸存的服务器获悉故障，它们就会扫描其目录分区和本地目录缓存，并清除位于故障服务器上的激活条目。由于 Actor 是虚拟的，因此当服务器发生故障时，不会有 Actor 发生故障。相反，对在故障服务器上激活的参与者的下一个请求会导致在幸存的服务器上创建新的激活。参与者的虚拟性质允许单个参与者的生命周期与托管服务器的生命周期完全解耦。服务器故障可能会也可能不会丢失该服务器上参与者的状态。奥尔良没有强制实施检查点策略。由应用程序决定需要对哪些参与者状态进行检查点检查以及检查频率。例如，参与者可以在每次更新其内存中状态后执行检查点，或者可以执行检查点并等待其确认，然后再将成功返回给其调用者。当服务器发生故障时，这样的参与者永远不会丢失其状态，并且当在不同的服务器上重新激活时，会以其最后的检查点状态重新恢复状态。然而，这种严格的检查点对于某些参与者来说可能太昂贵、太慢或根本没有必要。例如，代表设备（例如手机、传感器或游戏机）的参与者可能只是设备状态的缓存，设备通过向其参与者发送消息来定期更新该状态。没有必要对这样的演员进行检查。当服务器发生故障时，它将在不同的服务器上重新激活，并且其状态将根据稍后发送的数据重建

— 第 7 页 (英) / 译文 —

7 设备。如果应用程序能够承受偶尔丢失状态的小更新，另一种流行的策略是以固定的时间间隔定期检查参与者状态。检查点策略的灵活性，加上使用不同后端存储提供商的能力，使开发人员能够在应用程序的可靠性和性能之间实现所需的权衡。在某些情况下，用于路由消息的目录信息不正确。例如，本地缓存可能已过时并且具有不再存在的激活记录，或者取消注册激活的请求可能已失败。奥尔良不要求消息路由使用的目录信息完全准确。如果邮件被误导，收件人会将邮件重新路由到正确的位置，或者将邮件返回给发件人以进行重新路由。在任何一种情况下，发送方和接收方都会采取措施通过刷新本地缓存条目或更新参与者的分布式目录条目来纠正不准确性。如果目录丢失了对现有激活的跟踪，对该参与者的新请求将导致创建新的激活，并且旧的激活最终将被停用。3.9 最终一致性 在无故障的情况下，Orleans 保证一个 actor 仅具有一次激活。然而，当发生故障时，这只能最终得到保证。在服务器发生故障之后但在将其故障传达给所有幸存者之前，成员资格会不断变化。在此期间，如果发送者具有陈旧的成员资格视图，则注册激活请求可能会被错误路由。如果注册请求的目标不是其视图中目录分区的正确所有者，则该目标将重新路由该请求。然而，同一 actor 的两次激活可能会注册在两个不同的目录分区中，从而导致单个激活 actor 的两次激活。在这种情况下，一旦成员身份确定，就会从目录中删除其中一个激活，并向其服务器发送一条消息以将其停用。我们做出了这种权衡，倾向于可用性而不是一致性，以确保应用程序即使在成员资格不断变化的情况下也能取得进展。对于大多数应用程序来说，这种“最终单次激活”语义已经足够了，因为这种情况很少见。如果不足，应用程序可以依赖外部持久存储来提供更强的数据一致性。我们发现，以这种方式依赖恢复和协调比尝试保持目录的绝对准确性和本地目录缓存的严格一致性更简单、更稳健，并且性能更好。

3.10 消息传递保证 Orleans 通过在可配置的超时后重新发送未确认的消息来提供至少一次消息传递。可以通过保留已传递消息的标识符来添加“Exactly-once”语义，但我们认为成本过高，而且大多数应用程序不需要它。这仍然可以在应用程序级别实现。分布式系统中的普遍观点认为，维护消息之间的 FIFO

顺序成本低廉且非常理想。价格只是发送方、消息头和接收方队列中的序列号。我们最初的实现遵循该模式，保证从参与者 A 发送到参与者 B 的消息按顺序传递到

B，无论是否失败。然而，这种方法在具有大量参与者的应用程序中不能很好地扩展。角色对状态总共有 n^2 个序列号和队列。这种状态太多，无法有效维护。此外，我们发现大多数请求响应应用程序不需要 FIFO 消息排序。开发人员可以通过握手轻松表达逻辑数据并控制代码中的依赖关系，仅在收到对上一个调用的答复后才向参与者发出下一个调用。如果应用程序不关心两个调用的顺序，则会并行发出它们。4. 应用程序 Orleans

已被不同团队用作构建和运行多个云服务的平台，包括所有团队

Halo 4 视频游戏的云服务。本节介绍为两种不同游戏构建的三种服务。这些服务使用 Orleans 来实现游戏后端逻辑的不同部分，具有明显不同的使用模式和性能特征。大多数生产规模和性能数据都是保密的，因此我们报告了在生产前测试中在实验室中进行的测量。4.1 Halo 4 状态服务 状态服务负责跟踪所有活动的游戏会话、参与的玩家以及不断变化的游戏状态。它增强了玩家的配对体验，允许加入活跃的游戏，支持实时查看游戏会话以及其他功能。每个运行 Halo 4 的游戏机都会定期向该服务发出心跳调用，以报告其正在进行的的游戏状态。心跳调用的频率由服务控制，因此可以增加频率以获得更多交互体验，例如通过配套移动应用程序实时观看游戏。其他服务调用允许查询和加入实时会话，但我们将描述限制为仅心跳。在多人游戏会话中，每个控制台独立地将带有游戏状态更新的心跳消息发送到服务。游戏会话状态不会保存到持久存储中。它仅保留在内存中，因为基本事实始终位于控制台上，并且任何玩家只需进行一次心跳更新即可

— 第 8 页 (英) / 译文 —

8 发生故障时恢复游戏会话状态。心跳消息的有效负载包含压缩的会话数据，其中包括唯一的会话 ID、玩家 ID 和附加游戏数据。会话数据在处理之前必须被解压缩。游戏会话 Actor A Player Actor X Heartbeat 调用观察者 Game Session Actor B 游戏会话 Actor C Router Actor Player Actor Y Player Actor Z 通知 图 1：光环 4 在线服务 在线服务的结构如图 1 所示。有 3 种类型的参与者：路由器、游戏会话和玩家。来自控制台的传入心跳请求到达 Router actor，它解压缩数据、提取会话 ID，并将请求转发到正确的 Session actor。每个会话 ID 都有一个游戏会话参与者。Session actor 更新其内部内存状态，并定期（但不是每次心跳）使用从心跳数据中提取的玩家 ID 来调用 Player actor。玩家参与者还充当外部观察者（例如移动电话配套应用程序）的集合点，用于查找给定用户的当前游戏会话。观察者首先使用用户的 ID 作为密钥来调用用户的 Player actor。Player actor 返回对玩家当前正在玩的 Game Session actor 的引用。然后，观察者订阅直接从游戏会话参与者接收有关游戏会话更新的实时通知。由于 Router actor 是无状态的，Orleans 会动态创建此单个逻辑 actor 的多个激活，最多可达每台服务器上 CPU 核心的数量。这些激活始终位于接收请求的服务器本地，以消除不必要的网络跃点。其他三种 Actor 类型以单激活模式运行，随时有 0 或 1 次激活，并且它们的激活随机分布在所有服务器上。该服务的实现受益于参与者的虚拟特性，因为开发人员必须编写的调用参与者的代码相当简单：根据目标参与者的类型和身份创建对目标参与者的引用，并立即调用立即返回的参与者引用对象上的方法。无需编写代码来定位或实例化目标参与者并管理服务器故障。4.2 Halo 4 统计服务 统计是 Halo 4 的另一项重要服务。它处理已完成和正在进行的的游戏的结果以及重要事件的详细信息，例如成功射击、使用的武器、地图上事件的位置等。这些数据说明了玩家的成就、排名进展、个人统计数据、匹配等。该服务还处理有关玩家详细信息的查询以及游戏控制台和游戏网站发送的聚合。《光环 4》的统计数据非常重要，因为玩家不愿意失去自己的成就。因此，发布到服务的任何统计报告最初都会通过 Windows Azure 服务总线 [18] 可靠队列推送，以便在服务器发生故障时可以恢复和处理。图 2 显示了统计服务的简化架构，其中省略了许多辅助部分以节省空间。接收带有统计数据负载的 HTTP 请求的前端服务器将数据保存在 Azure 服务总线可靠队列中。一组单独的工作进程从队列中提取请求，并使用会话 ID 作为参与者身份来调用相应的游戏会话参与者。Orleans 将此调用路由到游戏会话参与者的激活，并在必要时实例化一个新参与者。游戏会话 Actor 首先将有效负载按原样保存到 Azure BLOB 存储，然后将其解压并将相关片段发送给游戏统计信息中列出的玩家的玩家 Actor。然后，每个 Player actor 处理其作品并将结果写入 Azure 表存储。该数据稍后用于提供有关玩家状态、成就等的查询。游戏会话参与者 A 游戏会话参与者 B 游戏会话参与者 C 玩家参与者 X 玩家参与者 Y 玩家参与者 Z 玩家参与者 U 玩家参与者 W 查询 BLOB 存储 表存储 保存游戏统计数据 保存玩家统计数据 发布统计查询

读取玩家统计数据 图 2：光环 4 统计服务 将游戏和玩家统计数据写入存储的操作是幂等的，因此在发生故障时可以安全地重放它们。如果请求无法处理（超时或因异常而失败），使请求出列的工作线程将重新提交该请求。奥尔良在游戏期间将游戏演员保留在记忆中。演员们处理和积累

— 第 9 页 (英) / 译文 —

9 部分统计数据保存在缓存中，以便在游戏结束时合并。同样，Player actor 在用于处理统计数据或提供查询服务时会保留在内存中，并缓存玩家的数据。在这两种情况下，缓存都会减少存储的 IO 流量并降低延迟。4.3 银河统治服务 银河统治是一款回合制、面对面的战术扩展和征服游戏，适用于手机和 PC。每个玩家提交一个游戏回合的战斗命令。游戏处理回合并前进到下一个回合。银河统治使用四种类型的参与者：有状态的游戏会话和无状态的视频管理器、管家和通知参与者。当从玩家处收到该回合的战斗命令时，每个游戏会话参与者都会执行游戏逻辑，并生成写入持久 Azure 存储的结果。然后，它向视频渲染服务发送请求，以生成一组短视频（最多 90 秒）。游戏回合命令 视频管理器 Actor 视频渲染系统 输入队列 输出队列 游戏回合结果 游戏会话数据 读取/更新游戏状态 游戏会话 Actor A 游戏会话 Actor B 游戏会话 Actor C 管家通知 Actor 图 3：银河统治游戏服务 每个游戏会话 Actor 都保存该游戏当前会话状态的缓存副本。游戏状态数据可能很大（数兆字节），并且需要一些时间从存储中读取和补充，因此系统将活动会话保留在内存中，以减少处理延迟和存储流量。随着时间的推移，Orleans 运行时可能会停用不活动的游戏会话。它们会在需要时自动重新激活。视频管理器参与者池负责向视频渲染系统提交作业，并在渲染作业完成时接收通知。由于这些参与者是无状态的，奥尔良运行时透明地创建它们的额外激活来处理增加的工作负载。参与者设置计时器来定期轮询已完成的渲染作业，并将其转发给通知参与者。一旦游戏回合完成并生成视频剪辑，通知参与者就会向设备上运行的游戏客户端发送一条消息。Housekeeper Actor 设置一个计时器，定期唤醒它以检测废弃的游戏并清理不再需要的持久游戏会话数据。4.4 数据库会话池 分布式系统中的一个常见问题是管理对共享资源（例如数据库、队列或硬件资源）的访问。一个示例是 N 个前端或中间层服务器共享对具有 M 个分片的数据库的访问权限。当每台服务器打开与每个分片的连接时，将创建 N×M

数据库连接。当 N 和 M 达到数百时，连接数量会激增，这可能会超出网络的限制，例如网络负载均衡器的每个 IP 地址的最大端口数。Orleans 提供了一种简单的方法来实现共享连接池。在此应用程序中，引入了参与者类型 Shard 来封装与数据库分片的打开连接。该应用程序没有打开直接连接，而是使用分片参与者作为向分片发送请求的代理。通过哈希将每个数据库分片映射到一个或几个分片参与者，应用程序可以完全控制分片参与者的数量，从而完全控制数据库连接。使用虚拟参与者实现连接池的另一个好处是代理的可靠性，因为它们会在服务器发生故障后自动重新激活。在这个场景中，Orleans 用于实现有状态连接池，以动态、可扩展和容错的方式共享对有限资源的访问。

5. 表现

本节我们研究奥尔良的表现。我们从针对系统特定部分的综合微观基准开始。接下来，我们报告运行第 4.1 节中描述的 Halo Presence 服务的生产代码的整个系统性能。综合微观基准测试运行 5 次，每次 10 分钟，生产性能评估运行每次 30 分钟。测量结果是

该测试在最多 125 台服务器组成的集群上执行，每台服务器配备两个运行频率为 2.10GHz 的 AMD 四核 Opteron 处理器，每台服务器总共有 8 个内核，32GB RAM，全部运行 64 位 Windows Server 2008 R2 和 .NET 4.5 框架。

5.1 综合微基准 异步 IO 和协作多任务处理

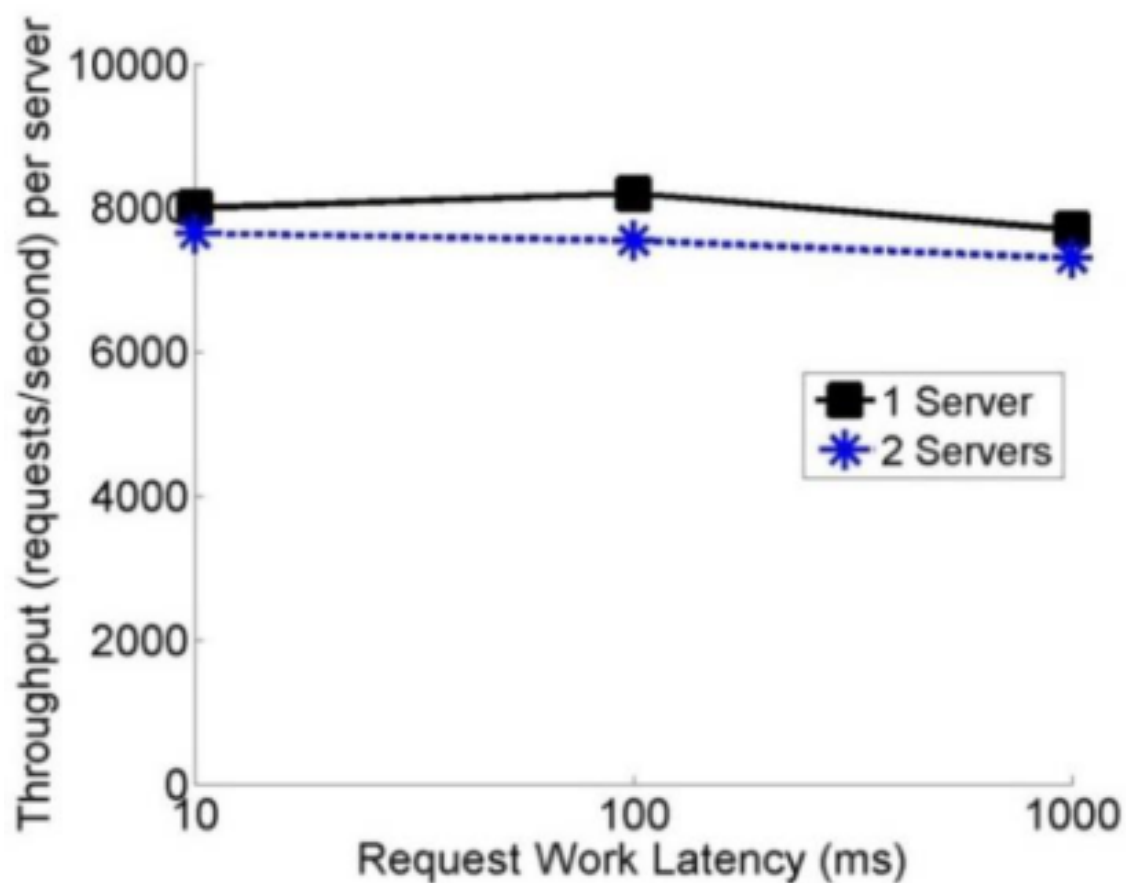
在此基准测试中，我们评估了异步消息传递与协作多任务处理的有效性。我们展示了这两种机制如何有效地掩盖参与者工作中的延迟。该测试使用 1000 个参与者，并从多个负载生成器发出请求以使系统完全饱和。每个请求都会模拟一个参与者向另一个参与者或外部服务（例如存储）发出远程调用的情况。我们改变延迟。由于远程调用是异步的，当前请求不会被阻塞，因此调用

— 第 10 页 (英) / 译文 —

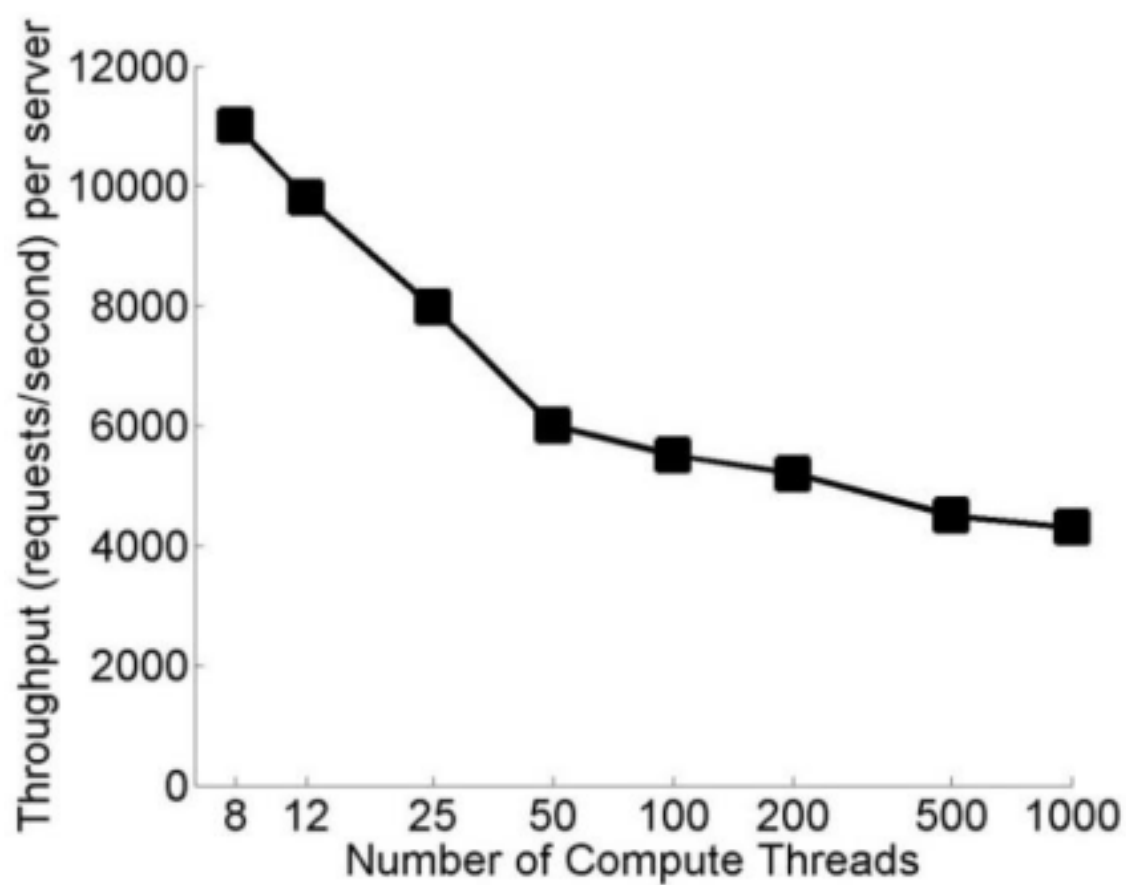
可以释放 10 个线程来处理另一个请求。图 4 显示，来自参与者内部的模拟外部调用的延迟增加对系统的整体吞吐量影响很小。图 4：使用异步 IO 和协作多任务处理进行延迟屏蔽 图 5：少量协作线程 协作多任务处理和线程 Orleans 调度程序使用少量计算线程（通常等于 CPU 数量）进行协作多任务处理。众所周知，这比使用大量线程更有效。我们使用短 ping 消息和 Orleans 调度程序使用的不同数量的线程来运行吞吐量测试。结果如图 5 所示。正如预期的那样，我们看到吞吐量稳步下降，因为线程上下文切换开销、额外内存、更长的操作系统调度队列和缓存局部性的减少导致线程数量增加。隔离的代价奥尔良中参与者的隔离意味着参与者调用中的参数必须被深度复制。在此基准测试中，客户端调用具有给定参数的第一个参与者，该参数调用具有相同参数的第二个参与者，一次按原样传递，一次作为不可变传递（意味着它不被复制）。在基准测试中，50% 的调用是本地调用，50% 是远程调用。一般来说，远程调用的比例越大，深度复制导致的吞吐量下降越小，因为序列化和远程消息传递的开销会增加。在数百台服务器上运行的大型应用程序中，大多数调用都是远程的，因此深度复制的价格将会降低。表 1 显示了三种数据类型的深度复制（请求吞吐量）的价格。对于一个简单的 byte[] 来说它非常小，大约为 4%。对于字典来说，复制的数据更多，但价格仍然低于 10%。对于复杂的数据结构，每个元素本身就是可变复杂类型的字典，开销会显著增加。参数类型描述 Don't Copy Copy % Decrease Byte[] 100 字节数组 7300 7000 4.3% Dictionary<int, string> 100 个元素的字典 6300 5700 9.5% Dictionary<int, List<int>> 100 个元素的字典，每个列表大小 1. 6500 3400 47.7 % 表 1：价格隔离性——具有不同参数类型的请求的吞吐量。图 6：Halo 4 Presence 服务的吞吐量。随着服务器数量的增加而线性扩展。

5.2 Halo Presence 性能评估 服务器数量的可扩展性

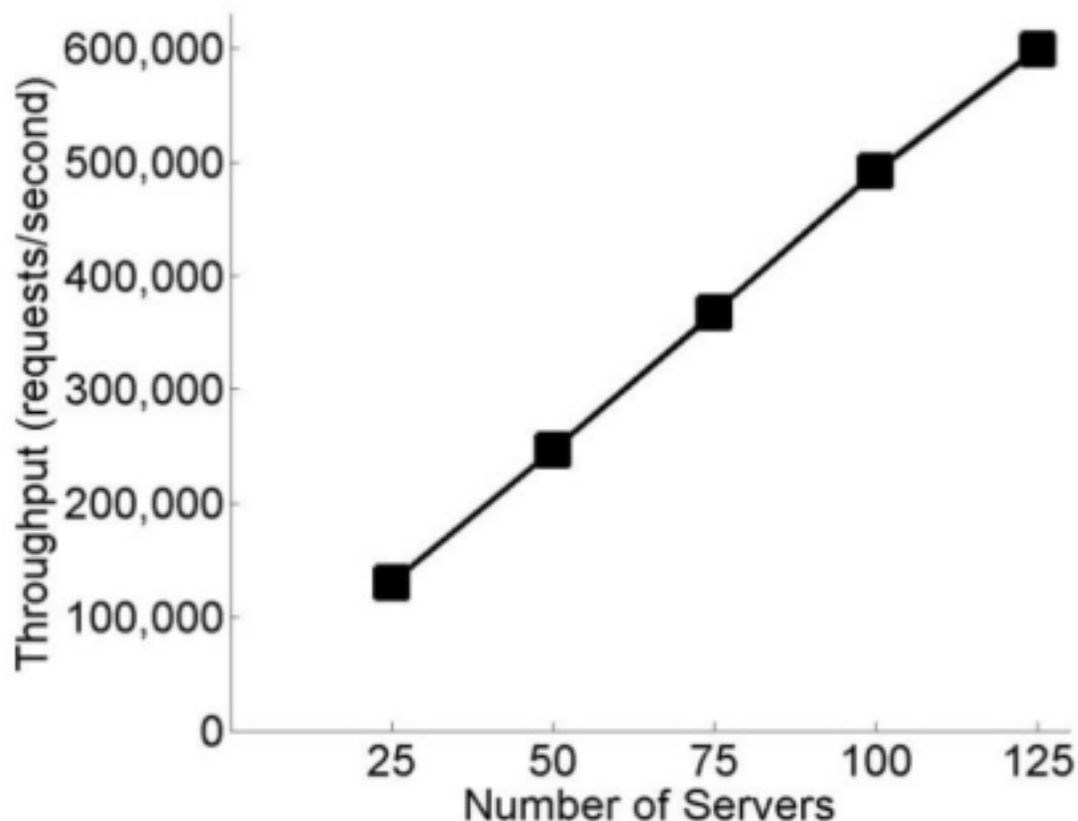
我们在测试集群中运行 Halo 4 Presence 服务的生产 Actor 代码，拥有 100 万个 Actor。我们使用足够的负载生成器来使奥尔良节点完全饱和和生成的心跳流量，并测量服务可以维持的最大吞吐量。在本次测试中，节点始终稳定运行在 95-97% 的 CPU 利用率。每个心跳请求至少会产生两个 RPC：客户端到 Router actor 以及 Router actor 到会话 actor。第一个呼叫始终是远程的，第二个呼叫通常是远程的，因为随机放置



图：第 10 页插图 1/3 (摘自英文版)



图：第 10 页插图 2/3 (摘自英文版)



图：第 10 页插图 3/3 (摘自英文版)

— 第 11 页 (英) / 译文 —

Halo 4 Session 的 11 位演员。我们在图 6 中看到，25 台服务器的吞吐量约为每秒 130,000 次心跳（每台服务器约 5200 次）。随着服务器数量增加到 125 台，该吞吐量几乎呈线性扩展。参与者数量的可扩展性在此测试中，服务器数量固定为 25 台，并且我们使用多个负载生成器使系统饱和。在图 7 中，我们看到随着参与者数量从 2000 个增加到 200 万个，吞吐量几乎保持不变。大量时的小幅退化是由于内部数据结构的大小增加造成的。图 7：Halo 4 Presence 服务的吞吐量。随着参与者数量的增加而线性扩展。图 8：延迟作为负载的函数 延迟作为负载的函数

我们测量了心跳调用的延迟。服务器数量固定为 25 台，我们通过增加负载生成器的数量来改变负载。在图 8 中，x 轴描述了 25 台服务器的平均 CPU 利用率。当 CPU 利用率高达 19% 时，中值延迟约为 6.5 毫秒 (ms)；当 CPU 利用率为 34% 和 55% 时，中值延迟将分别增长至 10 毫秒和 15 毫秒。回想一下，每个心跳都是 2 个 RPC 调用，包括 CPU 密集型 blob 解压缩。此外，一小部分心跳会触发我们上面的描述中省略的其他参与者。由于额外的跃点和额外的 CPU 密集型处理，这些心跳的延迟自然会更高。这有助于提高平均值、标准差和第 95 个百分位数。

6. 相关工作

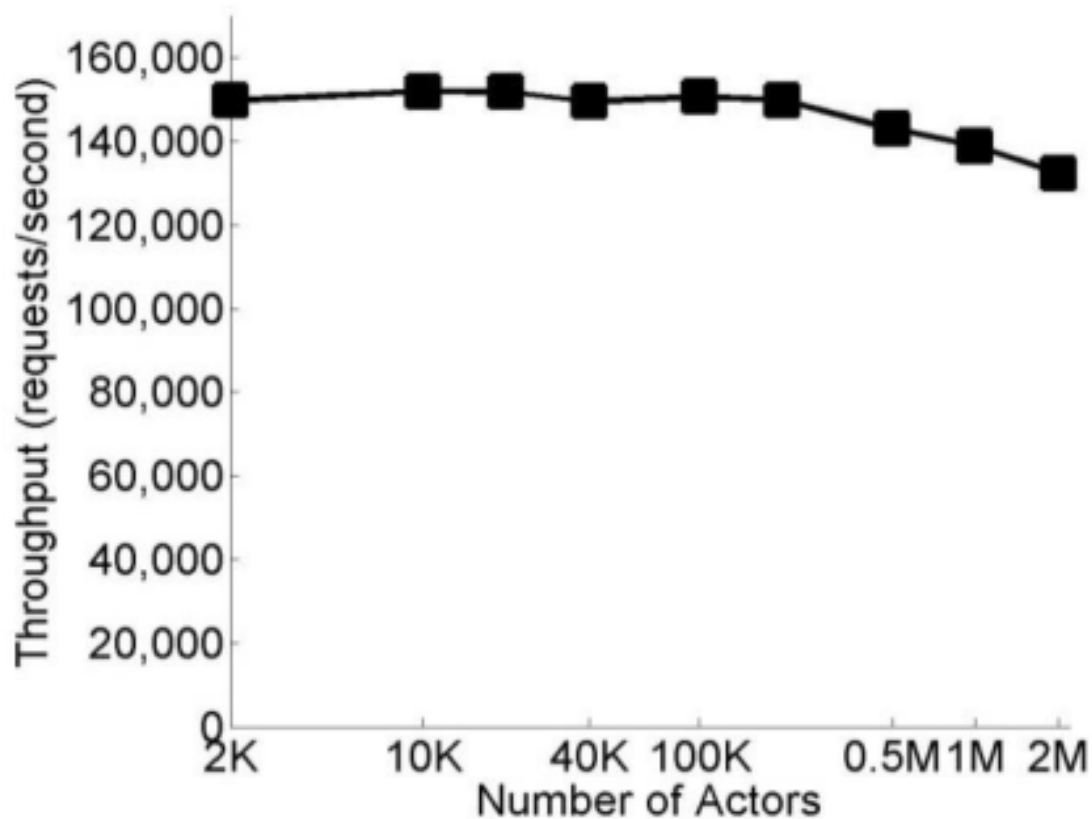
我们将 Orleans 与通用分布式编程框架和 Actor 框架进行比较。

6.1 分布式编程框架

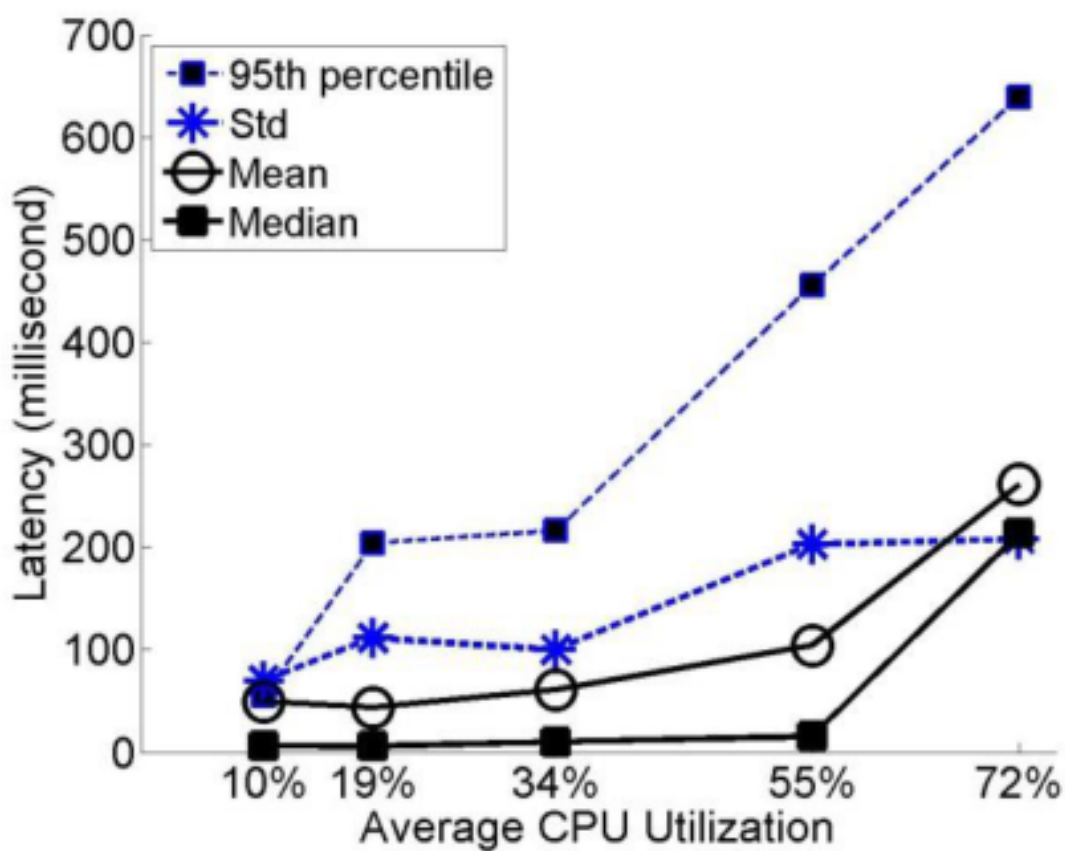
尽管 Orleans 在 Windows Azure 和 Windows Server 上运行，但几乎所有当前应用程序都使用 Azure。因此，它可与任何云应用程序开发框架相媲美。一种众所周知的框架是 Google App Engine (GAE)。尽管 GAE 和 Orleans 都提供面向对象的编程模型，但它们在两个主要方面有所不同。首先，GAE 的对象模型是 Java 或 Python 的对象模型，具有同步 RPC 和多线程。相比之下，Orleans 提供了一个具有异步 RPC 和单线程的参与者模型。其次，奥尔良对数据库服务不可知。相比之下，GAE 有一个带有事务的内置数据库服务。分布式对象模型（例如 Enterprise Java Beans (EJB)、分布式组件对象模型 (DCOM) 和公共对象请求代理架构 (CORBA)）与参与者框架有一些相似之处。与 Orleans 不同，它们主要基于同步通信，尽管有些也提供异步通信，例如 EJB 中的消息驱动 Bean。与 Orleans 不同，它们需要对象的静态放置，例如通过将类映射到服务器或类分区到服务器，并允许对象可以共享状态的多线程服务器。它们都没有提供虚拟演员的抽象。然而，它们确实提供了奥尔良之外的许多有用的功能：事务、可靠的消息传递、请求排队和发布-订阅。为了通过缓存提高多层架构的性能，已经做了很多工作（[9][11][13][14][16][19]）。然而，这将确保数据一致性和数据完整性语义的负担转移给了应用程序。像参与者模型这样的函数传送范例消除了这个问题。

6.2 Actor 框架

Orleans 结合了许多以前的 Actor 系统的技术。[7] 中参与者框架的比较确定了五个关键属性：状态封装、安全消息传递（通过深度复制按值传递）、位置透明性、移动性和公平调度。奥尔良完全支持前三项。它支持弱移动性——演员可以从一台机器移动到



图：第 11 页插图 1/2 (摘自英文版)



图：第 11 页插图 2/2 (摘自英文版)

12 另一个但不在处理请求时。它支持尽力而为的公平调度：在行为良好的应用程序中，每个参与者都会收到其公平份额的CPU时间。Erlang 是一种具有关联参与者模型的函数式编程语言 [3]。Erlang actor 称为进程。与奥尔良一样，每个参与者都是单线程的，通过逻辑引用访问，并通过单向消息与其他参与者通信。原则上，参与者只有私有状态，但实际上参与者经常在表或注册表中共享状态。与 Orleans 不同，Erlang actor 是显式创建的。生成操作在调用者的服务器（默认）或远程服务器（在参数中指定）上创建一个 Erlang 进程。进程创建后，其位置不能更改。这阻止了奥尔良中发现的重要优化：跨服务器的动态负载平衡、参与者迁移以及通过在其他服务器上重新启动其参与者来自动服务器故障处理。Erlang 应用程序通过使用链接操作显式控制错误如何从一个进程传播到另一个进程。如果一个进程没有链接到另一个进程并引发未处理的异常，它就会默默地死亡。相比之下，在奥尔良，异常通过 Promise 自动在分布式调用链中传播。开放电信平台 (OTP) 扩展了 Erlang 的运行，具有使应用程序免受容错、分布和并发方面影响的功能。为了启用特定于应用程序的错误处理，它有一个可选模块来跟踪监督树，这是由进程创建引起的进程树。它提供了两种处理子故障的选项：要么它的主管重新创建它，要么它的兄弟被杀死并且主管重新创建它们。虽然很灵活，但这需要开发人员显式管理每个参与者的生命周期。相比之下，在奥尔良，不存在创造等级制度。Actor 是由运行时自动创建和垃圾收集的。如果参与者的服务器发生故障，则会在不同的服务器上自动重新创建参与者。这种自动生命周期管理极大地简化了编程。Akka [2] 是一个基于 Actor 的 Java 和 Scala 编程框架。与 Orleans 一样，每个 Actor 都是单线程的，只有私有状态，并且通过逻辑引用进行访问。Akka 保证每对 Actor 之间最多一次消息传递和 FIFO 排序。与 Orleans 和 Erlang 不同，Actor 是在 Akka 中显式创建的，并且创建层次结构驱动异常处理。在 Akka 中，每个参与者在逻辑上由反映监督层次结构的路径表达式命名。Orleans 使用类类型和键。Akka 使用物理路径进行远程 Actor 引用。与在 Erlang 中一样，Actor 的位置在创建时是固定的，这会阻止动态负载平衡、Actor 迁移和自动处理机器故障。Akka 具有 Orleans 未涵盖的功能，例如在运行时将新代码加载到 Actor 中的能力以及确保一组 Actor 调用的效果是原子的的事务机制。然而，这些仅适用于同一台机器上的参与者，因此不适用于分布式参与者模型。[5] 中描述了奥尔良的原型。早期版本并不支持虚拟演员的所有方面。相反，它需要参与者的显式生命周期管理。它会在每次调用时自动保留参与者状态，这对于我们的生产用户来说太昂贵了。这引出了 2.6 节中的持久化机制。它对承诺和延续使用了更明确的语法，我们用 .NET 4.5 的更简洁的异步等待语法和修改后的 Orleans 运行时来支持它。它为多激活参与者提供了多主复制方案，我们放弃了它，因为它无法提供良好的性能，而且我们的用户发现它太复杂。[5] 中的测量仅针对微基准，而不是第 5 节中的大规模生产场景。其他 Actor 框架 - 还有各种其他 Actor 编程模型。Kilim [16] 专注于单节点执行，并使用线程切换来建模 actor

执行和参与者之间的通信。ActorFoundry [1] 在 Actor 之间使用同步发送/接收通信，而不是 Orleans 中使用的异步、基于连续的 API。Thorn [4] (和 Erlang) 使用松散类型的动态参与者接口，需要注意匹配发送者和接收者代码以确保正确的消息传递语义。Orleans 使用强类型接口，可以轻松进行编译时一致性检查。Monterey 是一个基于 actor 的 Java 框架 [12]。与在奥尔良一样，应用程序使用密钥来获取参与者引用。与奥尔良不同，它需要对参与者进行明确的生命周期管理。它允许同步通信（尽管它警告这可能会导致性能问题）和多线程参与者。奥尔良两者都不允许。7. 结论 在本文中，我们描述了 Orleans，一个用于构建可靠、高效和可扩展的云应用程序的框架。我们介绍了虚拟参与者抽象，展示了它对可编程性的好处，并讨论了实现。我们描述了 Orleans 的生产用途并对其性能进行了测量。Orleans 可以通过多种方式进行扩展，以简化对底层平台功能的访问并通过更多服务来丰富它们。我们列表中最重要的是用于消息传递、事件流、主副本复制和事务的恰好一次语义。尽管这些都是成熟的技术，但我们预计需要创新来使它们对普通开发人员来说足够可靠、高效和可扩展。

— 第 13 页 (英) / 译文 —

13 8. 参考文献 [1] ActorFoundry, <http://osl.cs.uiuc.edu/af/> [2] Akka 文档, <http://akka.io/docs/> [3] Armstrong, J.: Erlang. CACM, 53, 9 (2010 年 9 月), 68-75 [4] Bloom, B., Field, J., Nystrom, N., Östlund, J., Richards, G., Strnisa, R., Vitek, J. 和 Wrigstad, T. Thorn: JVM 上的鲁棒、并发、可扩展脚本。在 OOPSLA 2009, 117-136。 [5] Bykov, S., Geller, A., Kliot, G., Larus, J., Pandya, R. 和 Thelin, J.: 奥尔良：适合所有人的云计算。在 SOCC 2011, 16:1-16:14。 [6] Eker, J., Janneck, J. W., Lee, E. A., Jie, L., Xiaojun, L., Ludvig, J., Neuendorffer, S., Sachs, S. 和 Yuhong, X. 驯服：异质性 - 托勒密方法。过程。IEEE 91, 1 (2003 年 1 月), 127-144。 [7] Karmani, R. K., Shali, A. 和 Agha, G.: JVM 平台的 Actor 框架：比较分析。第七届国际会议记录。论Prog的原则和实践。Java, 2009, 1120。 [8] Liskov, B. 和 Shriram, L. Promises: 分布式系统中高效异步过程调用的语言支持。SIGPLAN 1988, 260-267。 [9] Memcached, <http://memcached.org/> [10] Microsoft .NET, <http://www.microsoft.com/net/> [11] Miller, M. S., Tribble, E. D. 和 Shapiro, J.: 陌生人之间的并发：E 中的编程作为计划协调。在过程中。国际症状。关于可信的全球计算，2005 年，施普林格，195-229。 [12] Monterey, <http://www.cloudsoftcorp.com/products/monterey/> [13] Ports, D.R.K., T. Clements, I. Zhu, S. Madden 和 B. Liskov: 应用程序数据缓存中的事务一致性和自动管理。OSDI 2010: 279-292 [14] Power, R. 和 J. Li: Piccolo: 使用分区表构建快速分布式程序。OSDI 2010: 293-306 [15] 协议缓冲区, <http://code.google.com/p/protobuf/> [16] Srinivasan, Sriram 和 Mycroft, Alan, Kilim: Java 的隔离类型参与者，欧洲面向对象编程会议，2008 [17] Stoica, I., Morris, R., Karger D.R., Kaashoek, M. F., Balakrishnan, H.: Chord: 用于互联网应用程序的可扩展点对点查找服务。SIGCOMM 2001: 149-160。 [18] Windows Azure 服务总线, <http://www.windowsazure.com/en-us/home/features/messaging/> [19] Windows Azure 缓存, <http://www.windowsazure.com/en-us/documentation/services/cache/>